1. Identify/Discover Use Cases

#	Use Case
1	Solve Puzzle
2	Read Puzzle File
3	Validate Input
4	Search for Solutions
5	Print Results

2. Provide a Brief Description for Each Use Case

Solve Puzzle

This use case allows a User to solve a Sudoku puzzle. The user specifies one or more puzzle files, and the system reads each puzzle, validates it, searches for all possible solutions using depth-first search with backtracking, and prints the results. If no solution exists, the system reports that.

Read Puzzle File

This use case allows the system to read a Sudoku puzzle from a text file. The file contains nine lines of nine characters each, using digits 1–9 for pre-filled cells and _ for blanks. The file system provides the raw file contents.

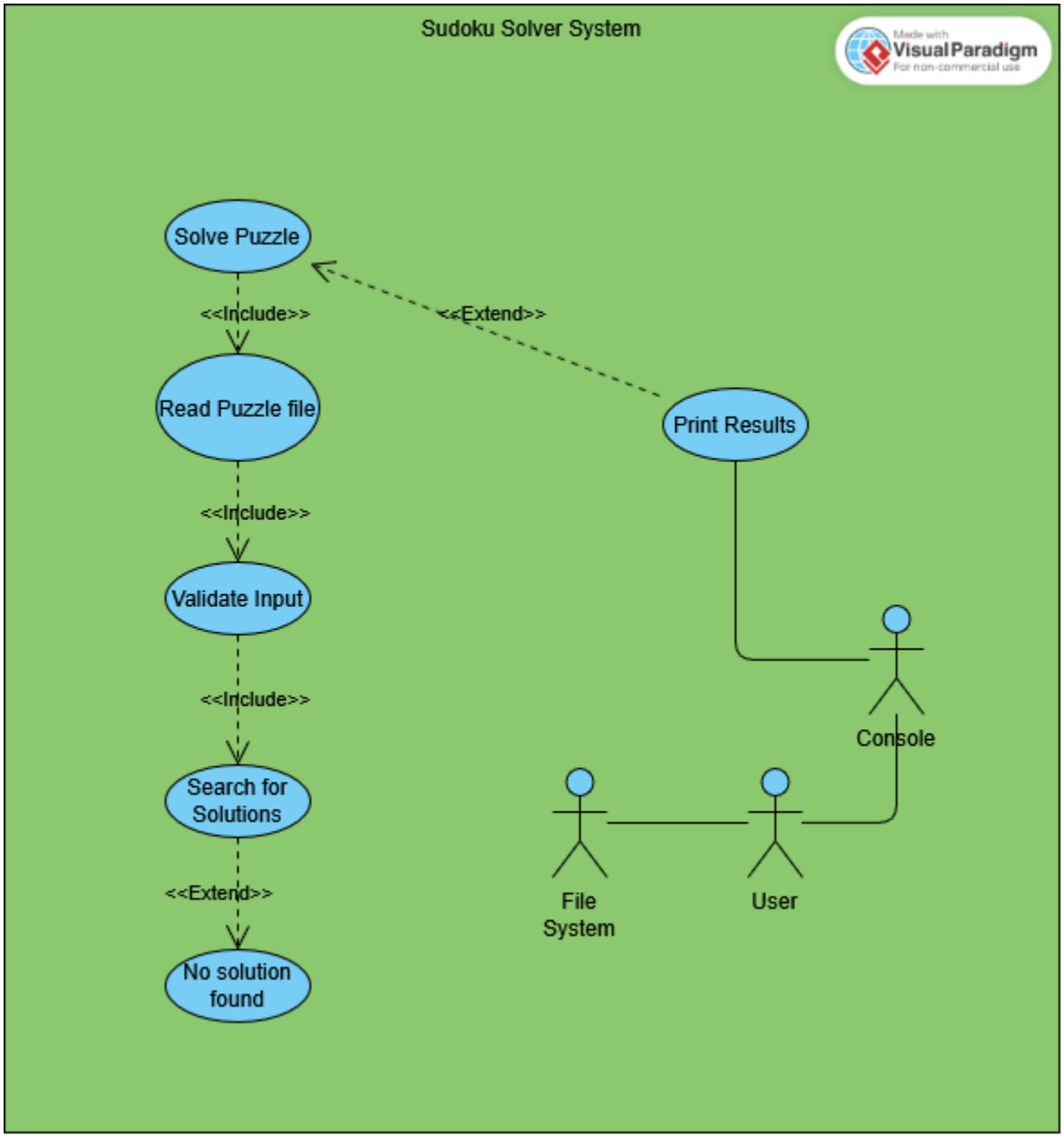
Validate Input

This use case allows the system to check that a puzzle is well-formed before attempting to solve it. It verifies that the file has exactly nine lines, each with nine cells, that every character is either _ or 1–9, and that no pre-filled digit conflicts with another in the same row, column, or 3×3 box.

Search for Solutions

This use case allows the system to find all solutions to a valid puzzle using recursive depth-first search with backtracking. The system tries digits 1–9 in each empty cell, checks validity against row, column, and box constraints, and backtracks when a dead end is reached. Every complete grid is stored as a solution.

Print Results

This use case allows the system to display the original puzzle, each solution found, or a "No solution found" message, along with error messages for invalid input. The Console receives the formatted output.

UML Case Diagram (Made with Visual Paradigm):